

Lab3实验报告

小组成员：2311858 许哲涵，2311576 赵宁，2313650 赵悦蛟

一、实验目的

- 1、掌握riscv 的中断相关知识
- 2、掌握中断前后如何进行上下文环境的保存与恢复
- 3、能够处理最简单的断点中断和时钟中断

二、实验内容

(一)本实验中重要的知识点

1.中断处理机制

- 中断是操作系统应对计算机系统突发状态变化（如程序异常、外设请求）的关键机制。当突发情况发生时，系统暂停当前正常工作，由操作系统接管并执行对应中断处理函数，处理完毕后返回原执行点继续运行，这一过程是操作系统实现并发、响应外部事件的基础。
- 以RISC - V架构为基础，涉及中断相关的寄存器（如用于控制中断使能、中断类型识别的寄存器）和指令，这些是硬件与操作系统中断处理交互的桥梁。

2.上下文环境的保存与恢复

- 为了保证中断处理前后程序能正确继续执行，需要保存和恢复上下文环境。实验中通过**中断帧（TrapFrame）**结构体存储要保存的各寄存器，还通过汇编代码实现上下文环境的保存与恢复机制。这体现了操作系统在进行任务切换或中断处理时，必须维护程序执行的状态一致性，确保多个任务或中断处理之间不会相互干扰。

3.中断类型的处理

- 实验中涉及**断点中断和时钟中断**的处理。时钟中断可用于实现系统的定时功能（如实验中每100次时钟中断打印信息并最终关机），体现了操作系统对时间资源的管理和基于时间的任务调度等理论；断点中断则用于程序调试等场景，是操作系统异常处理的一部分。

4.异常处理流程

- 从异常产生开始，涉及异常的捕获、上下文保存、异常类型识别、对应处理函数执行、上下文恢复等流程。这完整体现了操作系统处理异常的机制，保证系统在遇到异常时能有序处理，避免崩溃。

5.上下文切换相关的底层机制

- 实验中扩展练习涉及上下文切换的汇编代码操作（如 `csw sscratch, sp; csrrw s0, sscratch, x0`），这些操作是上下文切换底层实现的关键，体现了操作系统在任务切换时对处理器状态的精准控制，以实现多任务的并发执行。

6.异常类型的识别与处理

- 对于非法指令异常、断点异常等，操作系统需要能够捕获并进行相应处理（如输出异常类型和地址），这体现了操作系统的异常检测与容错能力，保证系统在遇到程序错误时能给出反馈并进行适当处理。

(二)OS原理中很重要，但在实验中没有对应上的知识点

- 系统调用完整实现：**系统调用是操作系统为用户态进程提供的、有控制地进入内核态以请求内核服务的唯一合法接口。其完整实现包含三个核心环节：首先是通过特殊的陷阱指令触发一个**软中断**，该中断会强制CPU将当前运行模式从权限较低的用户态 提升至权限最高的**内核态**，并跳转到预设的内核入口点。其次是**参数传递与解析机制**，在触发陷阱前，用户进程需按照约定将**系统调用编号**和调用参数存入指定通用寄存器（如RISC-V的a0-a6），内核陷阱处理程序会从这些寄存器以及用户栈中安全地提取并验证参数，防止恶意输入。最后是**系统调用表的设计与分发**，内核维护一个全局的**系统调用表**，该表将调用号映射到具体的内核服务函数地址，如 `sys_read`，`sys_write`，陷阱分发器根据获取的调用号，在此表中进行索引，并跳转到对应的服务函数执行。**实验中提到了系统调用概念，但是没有实际实现。**
- 进程隔离与内存保护：**该机制主要依赖硬件提供的**内存管理单元（MMU）**和**页表机制**来实现。关键在于**页表项中的权限位管理（R/W/X）**，每个虚拟内存页在页表中都有明确的权限标记，分别控制该页是否可读、可写、可执行。**实验中虽然启用了虚拟内存，但没有深入探讨权限保护。**
- 资源管理与回收：**其原理在于内核为每个进程维护一个**资源描述结构**，该结构记录了进程所拥有的全部资源。当进程通过 `exit` 系统调用正常终止或被信号强制终止时，内核会遍历该资源列表，执行**资源清理例程**：关闭所有打开的文件、释放占用的物理内存页、解除内存映射等。**实验中只关注了中断处理，没有资源管理概念。**
- 完整的权限级别管理：三个主要特权模式：用户模式（U-mode）、监督模式（S-mode）和机器模式（M-mode）**用户程序运行在限制最多的U-mode，无法执行特权指令，如操作页表寄存器 `satp`、直接访问外设等；操作系统内核运行在S-mode，可以执行这些特权指令，管理内存和设备；M-mode则拥有最高权限，用于引导系统和处理最底层的异常。**特权指令的执行控制**是权限管理的核心，CPU在当前模式下若尝试执行其不允许的特权指令，将触发非法指令异常。**模式切换**通常由异常、中断或 `ecall` 这类陷阱指令触发，硬件会自动提升模式并跳转到指定的异常处理程序；而通过 `sret` 等返回指令，CPU则从S-mode降权回到U-mode。**实验中只在S模式运行，没有真正的权限切换。**

练习1：完善中断处理（需要编程）

1.代码实现：

全局变量定义：

- `TICK_NUM` 定义为100，表示每100次时钟中断打印一次
- `num` 静态变量记录打印次数，初始值为0

```
1 #include <sbi.h>
2 #define TICK_NUM 100
3 static int num = 0;           // 打印次数计数器
```

在 `interrupt_handler` 函数的 `IRQ_S_TIMER` 分支中实现了核心逻辑：

```
1 clock_set_next_event(); // 设置下次时钟中断
2     ticks++;             // 计数器加一
3     if (ticks % TICK_NUM == 0) { // 每100次时钟中断
4         print_ticks();    // 输出"100 ticks"
5         num++;            // 打印次数加一
6         if (num == 10) {  // 打印10次后关机
7             sbi_shutdown();
8         }
9     }
```

结果：

```
ubuntu@ubuntu-VMware-Virtual-Platform: ~/labcode/lab3
Special kernel symbols:
  entry  0xffffffffc0200058 (virtual)
  etext  0xffffffffc0202078 (virtual)
  edata  0xffffffffc0207028 (virtual)
  end    0xffffffffc02074a0 (virtual)
Kernel executable memory footprint: 30KB
memory management: default_pmm_manager
physical memory map:
  memory: 0x0000000008000000, [0x0000000008000000, 0x00000000087fffffff].
check_alloc_page() succeeded!
satp virtual address: 0xffffffffc0206000
satp physical address: 0x00000000080206000
++ setup timer interrupts
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
ubuntu@ubuntu-VMware-Virtual-Platform:~/labcode/lab3$
```

实现要求的部分代码后，运行整个系统，大约每1秒会输出一次“100 ticks”，输出10行。

最后，输入make grade测试，得到结果：

2. 中断向量处理

- CPU跳转到 `stvec` 寄存器指定的异常处理入口 (`__alltraps`)
- 保存所有寄存器状态到陷阱帧 (`trapframe`)

3. 中断分发

```
1 void trap(struct trapframe *tf) {
2     trap_dispatch(tf);
3 }
4
5 static inline void trap_dispatch(struct trapframe *tf) {
6     if ((intptr_t)tf->cause < 0) {
7         interrupt_handler(tf); // 中断处理
8     } else {
9         exception_handler(tf); // 异常处理
10    }
11 }
```

4. 时钟中断识别

在 `interrupt_handler` 中:

- **中断原因解析:** 从 `tf->cause` 获取中断类型, 清除最高位得到纯原因码
- **中断类型匹配:** 匹配到 `IRQ_S_TIMER` (监管模式定时器中断)
- **设置下次中断:** 通过 `clock_set_next_event()` 重新编程定时器
- **系统时间更新:** `ticks` 全局变量递增
- **周期性输出:** 每100次中断调用 `print_ticks()` 输出信息
- **系统关机:** 输出10次后调用 `sbi_shutdown()` 安全关机

5. 中断返回

- 恢复之前保存的寄存器状态
- 执行 `sret` 指令返回到被中断的代码继续执行

总体来说, 产生一次时钟中断的执行流:

`set_sbi_timer()`通过OpenSBI的时钟事件触发一个中断, 跳转到 `kern/trap/trapentry.S` 的 `__alltraps` 标记 -> 保存当前执行流的上下文, 并通过函数调用, 切换为 `kern/trap/trap.c` 的中断处理函数 `trap()` 的上下文, 进入 `trap()` 的执行流。切换前的上下文作为一个结构体, 传递给 `trap()` 作为函数参数 -> `kern/trap/trap.c` 按照中断类型进行分发(`trap_dispatch()`, `interrupt_handler()`) -> 执行时钟中断对应的处理语句, 累加计数器, 设置下一次时钟中断 -> 完成处理, 返回到 `kern/trap/trapentry.S` -> 恢复原先的上下文, 中断处理结束。

扩展练习 Challenge1：描述与理解中断流程

1. ucore 中处理中断异常的流程：

1. 异常触发与硬件响应

当异常（如中断、指令错误等）产生时，CPU 硬件会自动完成部分现场保存（如将当前程序计数器 `epc`、状态寄存器 `status` 等存入特定寄存器），并根据异常类型跳转到对应的异常向量入口（由 `stvec` 寄存器指定，在 `idt_init` 中设置为 `__alltraps`）。

2. 异常向量表跳转

异常向量表由 `tools/vector.c` 生成，包含 256 个 `vectorN` 入口（对应不同异常类型）。每个 `vectorN` 会：

- 若需要，压入错误码（如部分异常需要额外信息）；
- 压入异常类型编号 `N`；
- 跳转到统一处理入口 `__alltraps`。

3. 现场保存（`__alltraps`）

`__alltraps` 是中断异常处理的统一入口，核心工作是通过 `SAVE_ALL` 宏保存所有通用寄存器（如 `ra`、`sp`、`a0-a7` 等）到栈中，形成 `struct trapframe` 结构（包含寄存器状态、`epc`、`cause` 等信息）。

4. 中断分发（`trap_dispatch`）

保存现场后，`__alltraps` 调用 `trap` 函数，并传入 `struct trapframe *` 作为参数（通过 `a0` 寄存器传递）。`trap` 函数调用 `trap_dispatch`，根据 `cause` 寄存器判断是中断（`cause < 0`）还是异常（`cause >= 0`），分别调用 `interrupt_handler` 或 `exception_handler`。

5. 具体处理

- 中断处理（`interrupt_handler`）：针对时钟中断、软件中断等，执行对应逻辑（如时钟中断更新 `ticks`、设置下次中断）。
- 异常处理（`exception_handler`）：针对非法指令、断点等异常，输出信息并调整 `epc`（如跳过错误指令）。

6. 恢复现场与返回

处理完成后，`trap` 函数返回，`__alltraps` 通过 `RESTORE_ALL` 宏从栈中恢复所有寄存器，最后通过 `sret`（或类似指令）从异常返回，继续执行被中断的程序。

`mov a0, sp` 的目的：

在 RISC-V 架构中，函数调用的第一个参数通过 `a0` 寄存器传递。`__alltraps` 中执行 `mov a0, sp` 的目的是：将栈指针 `sp` 的值（此时 `sp` 指向保存好的 `struct trapframe` 的起始地址）存入 `a0`，作为 `trap` 函数的参数（`struct trapframe *tf`），使 `trap` 函数能访问并处理异常现场信息。

2.SAVE_ALL中寄存器保存在栈中的位置的确定：

SAVE_ALL 中寄存器在栈中的保存位置由 `struct trapframe` 的定义（`trap.h`）确定。

```
1 struct trapframe {
2     struct pushregs gpr;    // 偏移量0: 通用寄存器
3     uintptr_t status;      // 偏移量32*REGBYTES: sstatus
4     uintptr_t epc;         // 偏移量33*REGBYTES: sepc
5     uintptr_t badvaddr;    // 偏移量34*REGBYTES: sbadaddr
6     uintptr_t cause;       // 偏移量35*REGBYTES: scause
7 };
```

具体来说：

- `struct trapframe` 包含 `struct pushregs gpr` 成员，而 `struct pushregs` 按固定顺序定义了所有通用寄存器（`zero`、`ra`、`sp`、`gp`、`t0-t6`、`s0-s11`、`a0-a7` 等）。
- `SAVE_ALL` 宏按 `struct pushregs` 的成员顺序依次将寄存器压入栈中，确保栈中布局与 `struct pushregs` 的内存布局完全一致，从而能通过 `sp` 指针直接访问 `struct trapframe` 中的各个字段。

3.对于任何中断，__alltraps 中都需要保存所有寄存器吗？

需要保存所有寄存器，理由如下：

1. 这些寄存器都将用于函数 `trap` 参数的一部分，如果不保存所有寄存器，函数参数不完整。如果修改 `trap` 的参数结构体的定义，可以不需要存所有的寄存器，比如0寄存器，因为它永远是0。
2. 上下文完整性：中断 / 异常可在程序执行的任意时刻发生（无论是用户态还是内核态），中断处理程序会修改寄存器的值。若不保存所有寄存器，被中断的程序恢复执行时，寄存器状态会被破坏，导致程序执行错误。
3. 无预知性：中断处理程序无法提前知道被中断程序正在使用哪些寄存器，必须假设所有寄存器都可能被使用，因此需全部保存。
4. 调试与恢复需求：`struct trapframe` 需完整记录异常发生时的寄存器状态，用于调试（如 `print_trapframe`）和正确恢复执行（如异常处理后调整 `epc` 并返回）。
5. RISC-V架构要求:硬件只自动保存少数CSR寄存器,并且通用寄存器的保存是软件的责任，这样可以确保 `sret` 能正确恢复到中断前的状态。

因此，`__alltraps` 必须保存所有寄存器以保证现场的完整可恢复性。

扩展练习 Challenge2：理解上下文切换机制

`csrw sscratch, sp`

- **操作：**将当前栈指针 `sp` 的值写入 `sscratch` 控制状态寄存器
- **目的：**临时保存用户态或内核态原始的栈指针
- **意义：**为后续保存寄存器到栈上做准备，因为马上要修改 `sp`

`csrrw s0, sscratch, x0`

- **操作：**

- 原子性地交换 `sscratch` 和 `x0` 的值
- 将 `sscratch` 的旧值（原始sp）读取到 `s0` 寄存器
- 将 `x0`（值为0）写入 `sscratch`
- 目的：
 - 获取原始sp值到通用寄存器 `s0`，以便保存到栈中
 - 将 `sscratch` 设为0，作为"内核模式"的标志

整体目的：

1. **栈指针保存**：确保原始sp值不会丢失，能正确保存到trapframe中
2. **模式标识**：设置 `sscratch=0` 表示当前运行在内核模式
3. **嵌套中断处理**：如果发生嵌套中断，通过检查 `sscratch` 可以知道当前上下文

save all里面保存了stval scause这些csr，而在restore all里面却不还原它们？那这样store的意义何在呢？

1. 保存 `stval`、`scause` 等 CSR 的意义

`stval`、`scause` 是 RISC-V 中记录异常关键信息的 CSR 寄存器：

- `scause`：存储异常 / 中断的原因（例如，时钟中断、非法指令、页错误等）。
- `stval`：存储与异常相关的辅助信息（例如，页错误时的错误地址、非法指令的指令编码等）。

保存 `stval`、`scause` 是为了让异常处理程序获取关键信息以完成处理。在异常处理流程中，`trap_dispatch` 等函数需要依赖这些信息判断异常类型并执行对应处理（例如，根据 `scause` 区分中断和异常，根据 `stval` 定位页错误地址）。SAVE_ALL 将这些 CSR 保存到 `struct trapframe` 中（如 `tf->cause` 对应 `scause`，`tf->badvaddr` 对应 `stval`），确保处理函数能访问到完整的异常上下文。

2. RESTORE_ALL 不还原的原因

`stval`、`scause` 是**异常本身的状态信息**，而非被中断程序的执行上下文。被中断的程序在恢复执行时，并不依赖这些寄存器的值：

- 被中断程序的正常执行只需要恢复通用寄存器（如 `ra`、`sp`、`a0-a7` 等）、`epc`（异常返回地址）、`status`（状态寄存器）等与自身执行相关的状态。
- `stval`、`scause` 的值仅对当前异常的处理有意义，处理完成后无需保留（下次异常发生时会被新的信息覆盖）。

因此，RESTORE_ALL 无需还原这些 CSR，只需还原被中断程序的执行上下文即可。

扩展练习Challenge3：完善异常中断

1.代码实现：

(1)异常处理逻辑实现（trap.c 中的 exception_handler 函数）

```
1  case CAUSE_ILLEGAL_INSTRUCTION:
2      // 非法指令异常处理
3      /*(1)输出指令异常类型 ( illegal instruction)
4      *(2)输出异常指令地址
5      *(3)更新 tf->epc寄存器
6      */
7      cprintf("Exception type: Illegal instruction\n");
8      cprintf("Illegal instruction caught at 0x%08x\n", tf->epc);
9      tf->epc += 4; // 跳过当前指令
10     break;
11 case CAUSE_BREAKPOINT:
12     //断点异常处理
13     /*(1)输出指令异常类型 ( breakpoint)
14     *(2)输出异常指令地址
15     *(3)更新 tf->epc寄存器
16     */
17     cprintf("Exception type: breakpoint\n");
18     cprintf("ebreak caught at 0x%08x\n", tf->epc);
19     tf->epc += 4; // 跳过当前指令
20     break;
```

- **识别异常类型：**通过 `tf->cause` 判断异常类型（`CAUSE_ILLEGAL_INSTRUCTION` 对应非法指令，`CAUSE_BREAKPOINT` 对应断点）。
- **输出异常信息：**分别打印异常类型（如 “Exception type: Illegal instruction”）和异常发生的指令地址（通过 `tf->epc` 获取，`epc` 寄存器存储异常发生时的指令地址）。
- **恢复程序执行：**异常处理完成后，需要更新 `tf->epc`（指令计数器），使其跳过当前异常指令（RISC-V 指令长度为 4 字节，因此 `tf->epc += 4`），确保程序能继续执行后续代码。

(2)异常触发测试（init.c 中的 kern_init 函数）

```
1  // 延迟测试异常，确保系统稳定运行
2      for (volatile int i = 0; i < 1000000; i++); // 简单延时
3
4      cprintf("Testing exceptions...\n");
5
6      // 测试1：非法指令异常
7      cprintf("1. Testing illegal instruction exception:\n");
8      asm volatile(
9          ".word 0x00000000 \n" // 非法指令
10         "j 1f \n"           // 跳转到标签1，如果异常处理正确会跳过这条指令
11         "1: \n"
12     );
13
14     // 测试2：断点异常
15     cprintf("2. Testing breakpoint exception:\n");
```

```

16     asm volatile(
17         "ebreak \n"           // 断点指令
18         "j 2f \n"             // 跳转到标签2，如果异常处理正确会跳过这条指令
19         "2: \n"
20     );
21
22     cprintf("Exception tests completed successfully!\n");

```

为了验证异常处理逻辑，需要在初始化流程中主动触发两种异常：

- **非法指令异常触发：**通过汇编指令 `.word 0x00000000` 插入一条非法指令（RISC-V 中无此编码的合法指令），强制触发非法指令异常。
- **断点异常触发：**通过 `ebreak` 指令（RISC-V 的断点指令）主动触发断点异常。

测试代码如下所示。

2.测试输出：

```

ubuntu@ubuntu-VMware-Virtual-Platform: ~/labcode/lab3
memory: 0x0000000008000000, [0x0000000008000000, 0x00000000087fffffff].
check_alloc_page() succeeded!
satp virtual address: 0xfffffffffc0206000
satp physical address: 0x00000000080206000
++ setup timer interrupts
Testing exceptions...
1. Testing illegal instruction exception:
Exception type: Illegal instruction
Illegal instruction caught at 0xc02000d8
2. Testing breakpoint exception:
Exception type: breakpoint
ebreak caught at 0xc02000ea
Exception tests completed successfully!
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
ubuntu@ubuntu-VMware-Virtual-Platform:~/labcode/lab3$

```

说明：

```
1 | Exception type: Illegal instruction
2 | Illegal instruction caught at 0xc02000d8
```

- 当执行到 `.word 0x00000000`（非法指令）时，触发非法指令异常，进入 `exception_handler` 的 `CAUSE_ILLEGAL_INSTRUCTION` 分支，输出上述信息。其中 `0xc02000d8` 是 `tf->epc` 的值，即非法指令在内存中的地址。

```
1 | Exception type: breakpoint
2 | ebreak caught at 0xc02000ea
```

- 当执行到 `ebreak` 指令时，触发断点异常，进入 `exception_handler` 的 `CAUSE_BREAKPOINT` 分支，输出上述信息。其中 `0xc02000ea` 是 `tf->epc` 的值，即 `ebreak` 指令在内存中的地址。

本次实验结束。